START HERE

From idea to a working proof of concept on Azure

A guide for people who know their subject well and want to test an idea with a real, working system, without committing to a full project first.

How to read this guide

Everything is explained twice. The **plain-language** layer needs no technical background. Switch to **With technical detail** (top of the page) and extra blue panels appear with settings, commands and checks for the person who will actually build it. Your choice is remembered, and the print button saves whichever view you have open.

RUNNING EXAMPLE

Throughout, we follow a made-up company building **LeaveLoop**, a simple tool for requesting and approving time off. Employees submit requests, managers approve or decline, and HR sees a summary. It is fictional, so nothing real or confidential appears anywhere in this guide.

The journey in seven steps

1. **Frame the question**
   What must this proof of concept (POC) show? (Part 1)
2. **Listen and design**
   Talk to the people involved, find the real problem, break it into parts. (Part 2)
3. **Choose and cost**
   Compare the options honestly and price the one you pick. (Part 3)
4. **Get access**
   Ask for the permissions early, because waiting is the usual delay. (Parts 4 and 5)
5. **Build a thin slice**
   One complete path from start to finish, before anything else. (Part 6)
6. **Put it on Azure**
   First a quick demo, then the properly secured version. (Parts 7 and 8)
7. **Show it and decide**
   Measure against the question you started with. (Part 9)

Who does what

Person	Role in the POC	In LeaveLoop
Sponsor	Owns the decision at the end; removes blockers	Head of HR
Subject lead (you)	Knows the process; shapes the idea; gathers insight; judges the result	HR operations lead
Builder	Creates and deploys the system	One developer, part time
Cloud / IT team	Grants access, sets rules, reviews security	Central IT
Real users	Try it and tell you the truth	5 employees, 3 managers

TECHNICAL DETAIL

One-page cheat sheet

Reference architecture: **Static Web App** (front end) → **App Service** (API, Python/Node/.NET) → **Azure SQL** (data), with **Key Vault** (secrets), **Blob Storage** (files), a **managed identity** (the API's password-less login) and **Application Insights** (monitoring).

```
az login ; az account set --subscription "<id>"
az group create -n $RG -l $LOC --tags CostCenter=<code>
az appservice plan create -g $RG -n $PLAN --is-linux --sku B1
az staticwebapp create -g $RG -n $SWA -l <region> --sku Free
az webapp create -g $RG -p $PLAN -n $APP --runtime "PYTHON:3.12"
az webapp config appsettings set -g $RG -n $APP --settings ENABLE_ORYX_BUILD=true
SCM_DO_BUILD_DURING_DEPLOYMENT=true
az webapp deploy -g $RG -n $APP --src-path deploy.zip --type zip --async true
# front end: build with the API address, then upload ./dist with the Static Web Apps CLI
```

Five traps: missing `ENABLE_ORYX_BUILD` (503), zip with Windows paths/line endings, CORS origin mismatch, a front-end API address baked in at build time, and role assignments you are not permitted to create.

What a proof of concept is, and is not

A POC answers one question: **is this idea worth investing in?** It is not a small version of the finished product.

POC

Short, focused, fake or non-sensitive data. Built to be judged, then kept, changed or thrown away.

Pilot

Real users, real data, limited scope, support in place. Only after a POC says yes.

Production

Everyone, full security, backups, monitoring, an owner on call. A project, not an experiment.

Six things to write down before any work starts

Decide	What good looks like	LeaveLoop
The question	One sentence a sponsor can say yes or no to	"Can employees and managers handle leave online faster, with a clear history of who approved what?"
Users and roles	Three to five roles, a handful of real people	Employee, manager, HR viewer
In and out of scope	A written "not now" list	In: request, approve, summary. Out: payroll link, mobile app, public holidays by country
Success measures	Numbers or observations you can check	Request-to-decision time; 4 of 5 testers say they would switch
Time box	2 to 4 weeks of building, then a decision	3 weeks + 1 week to test
Data	Fake or non-sensitive only, unless approved otherwise	Invented employees and requests

THE RULE THAT KEEPS A POC SMALL

If a feature does not help answer the question, it is out of scope. Write it on the "not now" list so nobody feels ignored.

PART 2

Ideation: from a vague problem to a clear plan

Most failed POCs solve the wrong problem. This part is about finding the right one, together with the people who live with it, and then cutting it into pieces small enough to build.

2.1 Who to talk to

Before the first conversation, list everyone who touches the process. Four groups matter most:

Group	Why they matter	LeaveLoop	Time
The people who do the work	They know the real steps, not the official ones	HR administrators who process requests	30 min each
The people who use the output	They show what is actually needed	Managers planning team cover	30 min each
The owner or sponsor	Defines success and constraints	Head of HR	45 min
Risk, security and IT	They can stop you late if you skip them early	Data protection officer, cloud team	30 min

Aim for 6 to 10 conversations. After about five on the same theme, you mostly hear repeats.

2.2 How to run a good conversation

- **Ask about the last time, not in general.** "Walk me through the last time you approved leave" gives facts; "How do you usually approve leave?" gives a tidy story.
- **Listen for workarounds.** A personal spreadsheet or a sticky note is a requirement nobody wrote down.
- **Ask to see the real files.** Looking at the actual form teaches more than a description of it.
- **Do not pitch your solution.** People then answer politely instead of honestly.
- **Ask "what happens then?" and "why?"** two or three times. Stay curious, not corrective.
- **Two people, one speaking.** One asks, the other writes down exact words.
- **Play it back.** Send a short summary within a day and ask "what did I get wrong?"

Question bank

1. Walk me through the last time you did this, from the first trigger to the last step.
2. Which step takes the longest? Which one do you dread?
3. Where do mistakes or delays usually happen? What did the last one cost?
4. What do you keep outside the official system (notes, personal files)? Why?
5. What do you check before you trust a number or a form?
6. If you could change one thing tomorrow, what would it be?

1. What decision do you make with this information? When do you need it?
2. What do you do today when it is late or looks wrong?
3. Who do you ask when you do not understand it?
4. What would make you trust it more?
5. What would you never want shown to others?

1. What would make you say "this was worth it" in six months?
2. What is the cost of the problem today, in time, money, risk or reputation?
3. What must not change? What is open to change?
4. Who else must be on board for this to work?
5. What would make you stop this effort?

1. What data classes may a POC hold, and where may they be stored?
2. Which regions, naming rules and tags are mandatory?
3. Which sign-in method must we use?
4. What reviews or approvals are needed before real users try it?
5. Who owns the system after the POC if it succeeds?

2.3 Making sense of what you heard

1. **Find the real problem with "five whys".** Ask why repeatedly until you reach a cause you could act on.

LEAVELOOP

Requests are approved late. Why? Managers do not see them. Why? They arrive by email among hundreds. Why? There is no single place that shows what awaits a decision. So the problem is visibility, not approval rules.

2. **Draw the journey as it happens today:** one row per step, with who does it, what hurts, and the evidence (a quote, a number, a file).
3. **Turn each pain into a "How might we…" question.** "How might we show managers what is waiting for them?" This keeps options open.
4. **Write the problem in one sentence** and test it with two interviewees. If they do not nod, keep going.

Step today	Who	Pain	Evidence
Employee emails the manager	Employee	No confirmation it arrived	"I ask in the corridor if she saw it"
Manager replies, forwards to HR	Manager	Lost among other mail	3 of 4 managers keep a personal list
HR updates the master spreadsheet	HR admin	Retyping, clashes with other versions	Two files named "final"

2.4 Design thinking in five minutes

Design thinking is a way of working that starts with people and tests ideas early and cheaply. It has five stages, and you move back and forth between them.

1 · Empathise

Understand people by listening and observing. (Part 2.1 to 2.2)

2 · Define

State the problem in one clear sentence. (2.3)

3 · Ideate

Generate many options before choosing one. Quantity first, judgement later.

4 · Prototype

Make a cheap version people can react to. (2.6)

5 · Test

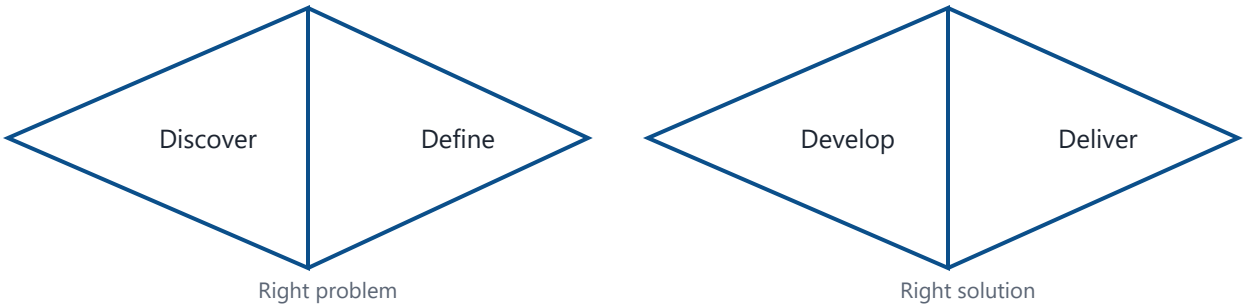
Watch real people use it; learn; adjust.

Repeat

The first answer is rarely the final one. Plan for two rounds.

The double diamond

A useful picture of the same idea: open up, then narrow down, twice. First to find the *right problem*, then to find the *right solution*.



A useful question: what job is it hired to do?

"Jobs to be done" asks why someone would use a tool, not what features it has. Managers do not want a leave system; they want to *plan team cover without chasing people*. The job shapes the priorities.

2.5 Breaking the idea into parts

Roles and what each may do

Role	Can see	Can change
Employee	Own requests	Create and withdraw own requests
Manager	Their team's requests	Approve or decline
HR viewer	Everything, as summaries	Nothing

The workflow as states

Draft → **Submitted** → **Approved** or **Declined**

For each arrow, write who may make the move. For each state, write who may change what. These two small tables become the rule book for the whole system.

User stories

One line per need: *As a [role], I want [action], so that [benefit]*. "As a manager, I want to see waiting requests in one list, so that I can plan cover."

Must, should, could, won't (this time)

Must

- Employee submits a request
- Manager approves or declines
- History of who did what and when

Should

- Email reminder to the manager
- HR summary by month

Could

- Calendar view of the team

Won't (this time)

- Payroll link, mobile app, holiday rules by country

The thin slice

Choose the smallest path that goes all the way through: *an employee submits, a manager approves, HR sees it.* Build that completely before adding anything. It proves the idea, exposes the real obstacles early, and always leaves you with something to show.

2.6 Cheap prototypes, early feedback

- **Paper or slides.** Sketch the main screens. Walk someone through them.
- **Clickable mock-up.** Linked slides or a design tool. Good for layout and wording.
- **"Wizard of Oz".** The screen looks real; a person does the work behind it. Tests whether the idea helps before you build the engine.
- **Working thin slice.** The real thing, minimal, with fake data.

Test with **three to five** people. Ask them to complete a task and stay quiet. Where they hesitate is where the design needs work. Write each finding down with the person's own words.

2.7 Working with the team

- **A one-page brief everyone has seen.** Template below. Update it when scope moves.
- **A weekly 20-minute show-and-tell.** Show what works; collect reactions; no slides.
- **A decision log.** Date, decision, why, who. It ends arguments about what was agreed.
- **Say "that's for later" kindly.** "Great idea; it's on the next-version list, and here's how we will judge when to do it."
- **Agree who decides what.** Who accepts scope changes? Who signs off the demo?
- **Share what you hear.** People contribute more when they see their input used.

```
POC BRIEF (one page)
Question this POC answers:
```

```
Users and roles:
Workflow (states and who moves them):
Data (what we store, fake or real):
Must / Should / Could / Won't:
Success measures:
Time box and key dates:
Decision owner:
Risks and open questions:
```

Date	Decision	Why	Who
12 Mar	Declined leave must give a reason	Employees asked for transparency	HR lead

TECHNICAL DETAIL

From the brief to a data model

Turn the nouns into tables and the states into a rule table. For LeaveLoop: `employees` (role, manager), `leave_requests` (employee, dates, status, decided_by, decided_at, reason), `audit_log` (who, what, old value, new value, when).

- **Key by the right thing.** If a status is stored "per employee", next year's request overwrites this year's. Key by what makes a record unique, usually unit *and* period.
- **Constraints in the database** (`CHECK` ranges, `UNIQUE` , foreign keys) catch mistakes the code misses.
- **Write the rule table first:** `(from_state, to_state) → roles allowed` and `state → role → writable fields` . Enforce it on the server.
- **Review the schema with a human before any screen is built.** It is the most expensive thing to change later.

PART 3

Choosing the approach, and what it will cost

Before building on Azure, check that building is the right answer. Then choose the simplest design that answers the POC question, and price it honestly.

3.1 The options on the table

Option	Good when	Watch out for
Improve what you have (better spreadsheet, shared form)	The pain is mostly habit or clarity	Hits a ceiling on history, permissions and scale
Buy a product	A standard tool already fits 80 percent	Licence cost per user; hard to bend to your process
Low-code platform (forms and flows)	Simple process, quick need, citizen developers	Limits on rules, data and cost at scale
Build on Azure	Your process is specific; you need control of data, rules and history	Needs a builder and an owner; you maintain it
Do nothing (for now)	The problem is small or the timing is wrong	The cost of waiting; make it a conscious choice

3.2 Compare them fairly: a weighted decision matrix

Pick the criteria that matter, give each a weight from 1 (minor) to 5 (critical), then score each option from 1 (poor) to 5 (excellent). The tool below does the arithmetic. **The example scores are placeholders, deliberately close; replace them with your own.** Do this with two or three colleagues, not alone, and discuss where you disagree: that is the useful part.

Decision matrix

Criterion	Weight	Improve	Buy a pl	Low-coc	Build or
Fit to our process	5	2	3	3	5
Time to first value	4	5	3	4	2
12-month cost (incl. people)	4	5	2	3	2
Security and data control	4	2	4	3	4
Skills we already have	3	5	4	3	2
Ease of changing later	3	2	2	3	5
Weighted score		**79 (69%)**	**69 (60%)**	**73 (63%)**	**78 (68%)**

Weights and scores run 1 to 5. Names are editable. Nothing leaves your browser.

3.3 What a POC really costs

THE BIGGEST COST IS USUALLY PEOPLE

A few weeks of a developer's and a subject lead's time typically cost many times more than the cloud bill for the same period. Include it, or the comparison is misleading.

What drives the Azure bill

- **Size of the hosting plan.** You rent capacity whether anyone is using it or not.
- **Database size.** Also billed while idle.
- **Extras for security and resilience.** Firewalls, private networking and geographic backups are valuable and can cost more than the app itself.
- **How many environments you run** (demo, test, production).
- **Forgotten resources.** Experiments left running are the most common surprise.

Free or very cheap tiers cover a POC well: a free static site, a small basic web plan, a small database. Plan to pay noticeably more only when real users and real data arrive.

Rough monthly cost estimator

Rounded planning figures in US dollars, to compare designs, **not a quote**. Prices vary by region, change over time and depend on usage. Confirm in the **Azure pricing calculator** before you commit.

API hosting (App Service): Basic B1 (~$13)

Front end (Static Web Apps): Free

Database: File database on the web app

File storage (Blob): None

Key Vault: None (settings only)

Monitoring: None

Edge protection: None

Private networking, threat protection, extras: None

Environments (e.g. 2 = demo + test): 1

Build and support hours (one-off): 160

Blended cost per hour (US$): 60

POC length (months): 3

$13	$39	$9,600	$9,639
cloud cost per month	cloud cost over 3 months	people time (one-off)	total POC cost

People time is about 246 times the cloud cost in this scenario. Figures are rounded planning estimates, not quotes.

3.4 Choosing between Azure alternatives

The principle: **choose the simplest service that answers the question, and leave the rest for later.** You can move up when you have evidence.

Decision	Simplest choice	Move up when
Where the screens live	Static Web Apps (free)	You need server-rendered pages or a gateway in front
Where the logic runs	App Service (one small plan)	Containers (Container Apps) for many services; Functions for event-driven tasks
Where the data lives	Demo: a file database on the web app. POC with real-ish data: Azure SQL	PostgreSQL if your team standardises on it; Cosmos DB for very flexible or global data
Where files go	Blob Storage (private)	Immutable storage when records must not change
How people sign in	A simple login (demo only)	Microsoft Entra ID before real data
How secrets are kept	App settings (fake data only)	Key Vault before real data

3.5 Using AI to help you choose

AI assistants are good thinking partners for this part of the work. Use them to widen and sharpen your thinking, and use people and official sources to decide and confirm.

Useful for

- Listing options you had not considered
- Drafting criteria and weights for you to adjust
- Explaining a pricing page or a service in plain words
- Building a first-pass cost model with its assumptions listed
- Challenging your choice: "what would make this fail?"
- Turning interview notes into themes

Be careful about

- Prices and limits: assistants can state them confidently and wrongly. Always confirm in the pricing calculator or with IT.
- Confidential data: do not paste personal or sensitive information; use the invented example.
- Out-of-date knowledge: products change.
- Authority: the assistant suggests; your team decides and owns it.

Prompts you can copy and adapt

```
I need to [one-sentence problem] for about [number] users in [type of organisation].
Constraints: [budget, timeline, data sensitivity, skills available].
List 5 realistic approaches (including improving what we have and doing nothing),
and for each give: when it fits, main risks, and what a 4-week test would look like.
Do not recommend one yet. Mark anything you are unsure about.
```

```
We are choosing between these options: [list].
Propose 8 evaluation criteria for a proof of concept (not a production system),
with a suggested weight from 1 to 5 and a one-line definition of what a score of 1, 3 and 5
means.
Point out criteria that overlap or that I might be missing.
```

```
Help me estimate the monthly cost of this design on Microsoft Azure:
[front end host, API host, database, storage, monitoring, security extras], region [region],
about [n] users, [n] environments.
List every assumption you make, give a low / expected / high range, and tell me which numbers
I must verify in the Azure pricing calculator. Do not present any price as final.
```

```
Here is our plan and the reasons we chose it: [paste].
Act as a sceptical reviewer. Give the 5 strongest reasons this could fail or be the wrong
choice,
the evidence that would convince you otherwise, and the cheapest test for each.
```

```
Below are anonymised notes from [n] interviews (no names, no confidential data):
[paste]
Group them into themes, quote the exact phrases that support each theme, note where people
disagree,
and list questions I should have asked but did not.
```

A GOOD HABIT

Keep a short note of what the AI suggested, what you verified, and what you changed. It shows your reasoning to the sponsor and protects you if a figure turns out to be off.

3.6 Guardrails against surprise costs

- Set a **budget alert** on day one at 50, 80 and 100 percent.
- **Tag** every resource with an owner and cost centre.
- Put a **review date** in the diary to shut down or scale down.

- Keep one named person responsible for the bill.

TECHNICAL DETAIL

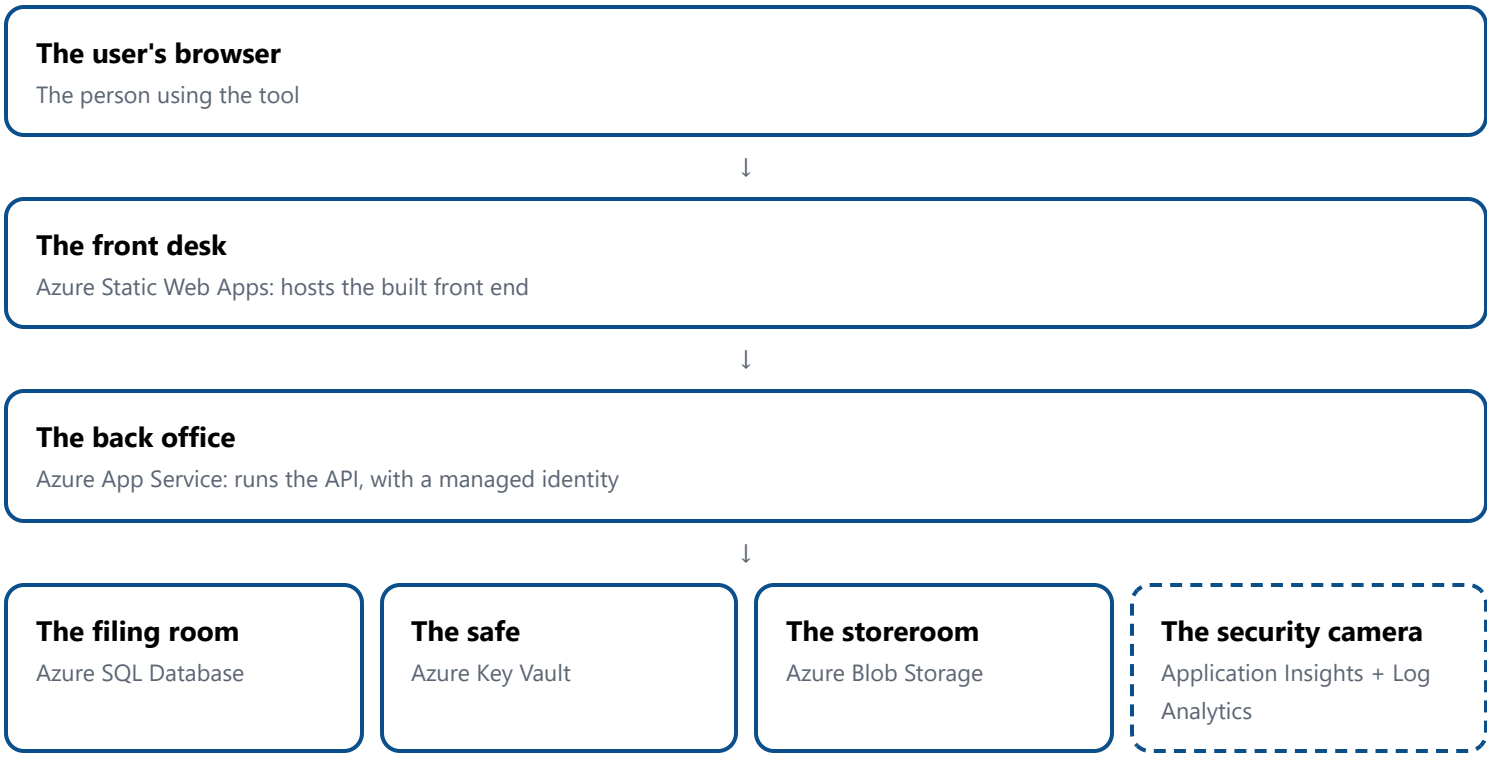
Pricing and budgets

- Use the Azure pricing calculator and pick the correct region. Check hours-per-month assumptions (730 for always-on).
- Budget alert and tag: `az consumption budget` or Cost Management in the portal; tag at creation with `--tags CostCenter=... Owner=...`.
- Cost drivers in order of size for this stack: App Service plan, SQL database, any edge/WAF or private networking, then monitoring ingestion. Storage and Key Vault are small at POC volumes.
- Log Analytics bills by data ingested; keep retention short for a POC.

PART 4

How the pieces fit together

A web application, however modest, is a small team of specialists. Knowing who does what makes every later conversation with IT easier.



Piece	Plain meaning	Why it matters
Subscription	The billing account	Costs and limits are tracked here
Resource group	A folder holding everything for the project	Delete the folder to remove everything
Region	Which data centre	Organisations often restrict this
Managed identity	A staff badge for the application itself	The app can enter the filing room and the safe without a password anyone could steal
Role (access control)	What a badge opens	Give the least needed
Tags	Labels on resources	Cost tracing and ownership

TECHNICAL DETAIL

Reference services

Service	POC setting	Notes
Static Web Apps	Free	Built files uploaded with a deployment token
App Service (Linux)	B1, Python 3.12 (or Node/.NET)	HTTPS only, TLS 1.2, FTPS disabled, health check path, system-assigned identity
Azure SQL Database	S0; Entra-only authentication	Optional ledger tables for tamper-evident audit data
Key Vault	Standard; role-based access	Soft delete on
Storage account	Standard LRS; private containers	No public blob access, no shared keys, versioning and soft delete
Application Insights	Linked to a Log Analytics workspace	Short retention for a POC

Keep every environment difference (database address, secrets, allowed origins, demo-data switch) in environment variables read in **one config file**. The same code then runs locally, in the demo and in production.

PART 5

Getting access, and why to ask early

The most common reason a POC slips is not the building. It is waiting for permission.

Inside an organisation's cloud, three levels of permission matter, and the person building usually has only the first:

1 · Create things

Make the web app, the database and the storage. Usually granted for one project folder.

2 · Let things talk to each other safely

Allow the application to open the safe and the filing room without a password. Needs a higher permission that IT guards.

3 · Organisation-wide settings

Switching on services for the whole subscription. Held by very few people.

A POC can proceed with level 1 alone, using simpler settings and fake data. Levels 2 and 3 are needed before anything real.

Send these questions to IT on the first day

Copy, paste, and send. Do not wait until you need the answers.

```
Hello,

We are starting a short proof of concept [one-line description] using fake data only.
Could you help with the following?

1. Which subscription and resource group should we use, and which regions are approved?
2. Are there mandatory tags (for example cost centre and owner) or a naming standard?
3. Are Azure Static Web Apps allowed, or must the front end sit behind another gateway?
4. Can I be given permission to create role assignments, or who can create them for us?
5. Are private networking or a web application firewall required even for a POC?
6. Which sign-in method must we use if real users are involved?
7. Are there limits on what data a POC may hold?
8. Is my access time-limited (activated per session)? If so, how do I activate it?

Our target is to have a demo ready by [date]. Many thanks.
```

ALLOW FOR TIME-LIMITED ACCESS

Some organisations grant powerful roles only for a few hours at a time, activated on request. If something that worked in the morning is denied in the afternoon, the role has probably expired; activate it again.

TECHNICAL DETAIL

Roles and tools

To do this	You need
Create resources in a resource group	Contributor on the group
Create role assignments (managed identity to Key Vault / Blob)	Owner, User Access Administrator or RBAC Administrator on the group
Write Key Vault secrets and blob data	Key Vault Secrets Officer; Storage Blob Data Contributor (data-plane roles, separate from Contributor)
Register resource providers	Contributor/Owner at *subscription* level
Use Microsoft Graph (`az ad ...`)	Allowed by conditional access; often blocked on managed laptops

Tools: Azure CLI, Git, Node 20+, your language runtime. Open a **new terminal** after installing each so it is found.

If database tokens are blocked on your laptop (conditional access), use the in-app setup route described in Part 7, stage 2. **If Graph is blocked**, read your object id from the `oid` claim of your Azure Resource Manager token.

```
az login
az account set --subscription "<id or name>"
az account show -o table      # confirm tenant and subscription before changing anything
```

PART 6

Building it: the order that works

Build foundations first. Each step ends with something a non-technical colleague can review.

1. **The records**
   Decide what is stored and how records relate. Review it with the subject lead. This is the hardest thing to change later.

2. **The rule book**
   Write who may do what, in which state. Put the rules in one place the whole system obeys.

3. **The history**
   Every change is recorded: who, what, before, after, when. Nothing is silently overwritten.

4. **Who is who**
   A simple sign-in for the POC, shaped so a proper one can replace it later.

5. **The main journey**
   The thin slice from Part 2, then other roles.

6. **Safety nets**
   Automatic tests for the rules that would quietly corrupt data if wrong, including checks that forbidden actions are refused.

7. **Demo data**
   An invented dataset that covers every state, so any reviewer sees every screen in a minute.

LEAVELOOP

Records: employees, requests. Rules: only a manager can approve, only their own team, only a submitted request. History: "Priya approved Sam's request on 4 March". Tests: an employee cannot approve their own request. Demo data: requests in every state.

WHY RULES MUST LIVE ON THE SERVER

Hiding a button is courtesy, not security. Anyone can send a request to the system directly, so the system itself must refuse what is not allowed.

Deploy an empty skeleton in week one. Problems with permissions, regions and policy show up early, when the app is empty, not the night before the demo.

TECHNICAL DETAIL

1. **Schema first** (SQLite locally, Azure SQL in the cloud). Plain SQL in a portable subset (`?` placeholders, no `ON CONFLICT` / `MERGE`); one small file that knows the dialect differences; a test that fails if the two schema files drift.
2. **Rule table**: `TRANSITIONS: (from, to) → roles` and `EDITABLE: state → role → fields` , validated in the API for every write. Return 403 for role problems and 409 for state problems.
3. **Audit**: one row per changed field, written in the **same transaction** as the change; if nothing changed, write nothing. Append-only (ledger table on Azure SQL).
4. **Auth**: signed, expiring tokens; keep `current_user` and `require_roles` as the only things routes depend on, so Entra ID can replace the login.
5. **Tests**: rules, permissions (including the must-be-refused cases), scoring or calculations. Run on every change.
6. **Seed script**: idempotent, gated by a setting (`SEED_DEMO_DATA`), only on an empty database.
7. **Operations**: `/health` and `/health/db` from day one; configuration in environment variables; a single entry point for database access.

PART 7

Putting it on Azure, in two stages

First get something clickable quickly. Then make it properly secure. Do not skip the first stage, and do not mistake it for the second.

Stage 1 · Demo

Goal: a public link people can click this week.

- Fake data only
- Simple shared login
- Data held in a simple file on the web app
- Secrets stored as settings

Good for: feedback sessions and the first demo.

Stage 2 · Proper

Goal: safe enough to trust with real information.

- A managed database with change history
- The application signs in with its own identity, no passwords
- Secrets in a vault
- Monitoring and a tested backup

Needed before: any real data or real users.

What happens in Stage 1, in plain terms: create a project folder; rent a small server for the back office; create a free place for the screens; give the back office its settings; upload the code; build the screens pointing at the back office; upload them; check that the health page says OK; try each role.

What changes in Stage 2: a database is created, an administrator sets up its tables once, the application is given permission to read and write rows (but not to change the structure), secrets move to the vault, and the temporary demo

shortcuts are removed.

TECHNICAL DETAIL · STAGE 1

Variables

```
$RG   = "rg-leaveloop-poc"
$LOC  = "<approved region>"
$PLAN = "plan-leaveloop-poc"
$APP  = "app-leaveloop-poc-<unique>"    # becomes <APP>.azurewebsites.net, globally unique
$SWA  = "swa-leaveloop-poc"
```

1 · Group and plan

```
az group create -n $RG -l $LOC --tags CostCenter=<code> Owner=<you>
az appservice plan create -g $RG -n $PLAN --is-linux --sku B1
```

2 · Static Web App and token

```
az staticwebapp create -g $RG -n $SWA -l <swa region> --sku Free
$swaHost  = az staticwebapp show -g $RG -n $SWA --query defaultHostname -o tsv
$swaToken = az staticwebapp secrets list -g $RG -n $SWA --query properties.apiKey -o tsv
```

The token is a password. Do not commit it.

3 · The API web app

```
az webapp create -g $RG -p $PLAN -n $APP --runtime "PYTHON:3.12"
az webapp config set -g $RG -n $APP --startup-file "bash startup.sh"
az webapp config appsettings set -g $RG -n $APP --settings `
  SCM_DO_BUILD_DURING_DEPLOYMENT=true ENABLE_ORYX_BUILD=true `
  "SECRET_KEY=<long random string>" "ALLOWED_ORIGINS=https://$swaHost"
```

`ENABLE_ORYX_BUILD=true` makes App Service install your requirements during a zip deploy. Without it the site returns 503.

4 · Deploy the API

Build the zip with the code, `requirements.txt` and the startup script at the **root**, forward-slash paths and LF line endings (use a script with Python's `zipfile`, not `Compress-Archive`).

```
az webapp deploy -g $RG -n $APP --src-path ./deploy.zip --type zip --async true
# poll https://$APP.azurewebsites.net/health  (first start installs packages: a few
minutes)
```

5 · Build and publish the front end

```
cd frontend
$env:VITE_API_URL = "https://$APP.azurewebsites.net/api"   # baked in at build time
npm ci
npm run build
npx --yes @azure/static-web-apps-cli deploy ./dist --deployment-token $swaToken --env
production
```

Anything with a `VITE_` (or similar) prefix is shipped to every browser: never put secrets there. Search the built bundle for the host name to confirm it points where you meant.

6 · Verify

- `GET /health` is OK; sign in as each role; try one allowed and one forbidden action.
- Network error in the browser but the API answers in `curl`: check CORS (`ALLOWED_ORIGINS` must hold the front end's exact URL).

```
az webapp log config -g $RG -n $APP --docker-container-logging filesystem
az webapp log tail   -g $RG -n $APP
```

For a throwaway demo, keep the SQLite file on the web app's persistent `/home` disk (for example `/home/data/app.db`). It does not survive scaling to several instances, so move to Azure SQL as soon as the data matters.

TECHNICAL DETAIL · STAGE 2

Database design

- **Microsoft Entra-only authentication** on the SQL server: no SQL logins exist.
- A named person (ideally a group) is the SQL admin.
- The API's **managed identity** is a database user with **read and write on tables only**. It cannot change the schema.
- Optional **ledger** audit tables: `WITH (LEDGER = ON (APPEND_ONLY = ON))`. Tampering becomes *detectable*, not impossible; the proof is only as strong as where the digests are kept.

One-off table setup

Option A, from your machine (if developer machines may obtain database tokens): add your IP to the server firewall, `az login --scope https://database.windows.net/.default`, run your setup script as the admin, create the API's user, then remove the firewall rule.

Option B, from inside the web app (when conditional access blocks A):

1. Make the web app's identity the **temporary** SQL admin (a management call, no database sign-in):

```
az sql server ad-admin create -g $RG -s <sql server> --display-name $APP --object-id
<app principal id>
```

2. Run setup at the app's start, controlled by a setting such as `SETUP_MODE=real`: create tables, seed reference data, grant the identity read/write. If `CREATE USER ... FROM EXTERNAL PROVIDER` fails ("Server identity is not configured"), create the user from the identity's **client id** (`CREATE USER [name] WITH SID = ..., TYPE = E`).
3. Watch the setup log until it reports success.
4. **Restore the human admin** and **delete the setup settings**; restart.
5. Confirm `/health/db` reports the Azure SQL database.

Rehearse on a throwaway database whose name ends in `_test`, and make any destructive reset command refuse to run elsewhere.

Connection

Set the server and database name as app settings; use the `ActiveDirectoryMSI` authentication mode on Azure (`ActiveDirectoryDefault` locally after `az login`). With no server configured, fall back to SQLite so local development and tests need nothing.

Secrets

Stage	Where secrets live
Local	Environment variables or an untracked `.env`
Demo, fake data	App settings, typed by the owner, never in the repo
Anything real	Key Vault, referenced from app settings

Reference syntax: `@Microsoft.KeyVault(VaultName=<vault>;SecretName=<name>)` ; the API's identity needs Key Vault Secrets User. **Verify it resolved**, otherwise the app silently uses the literal text as its secret:

```
az rest --method get --url
"https://management.azure.com/subscriptions/<SUB>/resourceGroups/$RG/providers/Microsoft.W
eb/sites/$APP/config/configreferences/appsettings?api-version=2022-03-01" --query
"value[].{setting:name,status:properties.status}" -o table
```

If the status is `AccessToKeyVaultDenied` after the role exists, re-running the same deployment (which re-saves the settings) fixes it; a plain restart may not.

Infrastructure as code (optional but recommended)

Describe resources in a Bicep file with one parameter file per environment; run a dry run first.

```
az deployment group what-if -g $RG -f main.bicep -p poc.bicepparam   # lists changes,
makes none
az deployment group create  -g $RG -f main.bicep -p poc.bicepparam
```

- Generate names from a prefix, the environment and a short hash of the resource group id.
- Apply the same tags to everything through one parameter.
- Guard role assignments with a switch (`createRoleAssignments`) so the template still deploys when you may not create them.
- Look up built-in role ids: `az role definition list --name "Key Vault Secrets User" --query "[].name" -o tsv` .
- What-if cannot see quota limits; if a deployment fails on size or region, change the parameter and re-run (it is declarative).

Keeping it safe

A POC with fake data can be simple. The habits you form now decide how easily it grows up.

Principles

- **Least access.** Everyone and everything gets only what it needs.
- **No secrets in the open.** Never in code, chat, slides or tickets. The person who owns a password types it in.
- **The system enforces the rules,** not the screen.
- **Keep a record.** Every important change says who, what and when.
- **Fake data until approved.**

What "tamper-evident" means

A tamper-evident record is like a tamper-proof seal on a jar. It does not stop someone opening the jar, but you can tell if they did. It is a strong help to trust, and it works best alongside tight permissions about who can write at all.

Before any real data arrives

Item	Why
Proper company sign-in (and a plan for outside partners)	Replace shared passwords; tie identities to roles
Secrets in a vault, with access granted by role	No secrets in settings
Database not open to the public internet	Remove the public path
Edge protection and a proper address (optional, by policy)	Defence at the front door
Locked storage for change-history proofs	Makes the tamper-evidence credible
Uploaded files in private storage with limited-time links	Out of the database; versioned
Backups across regions and a restore you have actually tested	Recovery
An owner, a runbook, and alerts	Someone is responsible
Data classification approved	Compliance

TECHNICAL DETAIL

Baseline in the POC

- HTTPS only, TLS 1.2, FTPS disabled; health check path set.
- Roles enforced on the server by role *and* workflow state; tests for the refusals.
- Audit log in the same transaction; ledger on audit tables if using Azure SQL.
- Managed identity for Azure-to-Azure access; no passwords in code.
- Storage private, no shared keys; uploads limited by size and an allow-list of types (never executables or HTML).
- Front-end build variables are public: no secrets.

Hardening list

- Entra ID sign-in (external tenant for partners).
- Private endpoints or a restricted SQL firewall; remove "allow Azure services".
- Front Door with WAF; custom domain.
- Immutability (time-based retention) policy on the ledger digest container.
- Blob uploads via short-lived links; geo-redundant backups; staging slot (Premium plan).

PART 9

Showing it, judging it, deciding

A POC exists to be judged. Plan the demo and the decision as carefully as the build.

The demo

1. **Start with the question,** in the sponsor's words, and the success measures you agreed.
2. **Show the old way in 60 seconds,** with a real example of the pain.
3. **Walk the thin slice as each role,** in the order a real request travels.
4. **Show the history.** This is often what wins trust.
5. **Show a refusal:** someone trying something not allowed, and the system saying no.
6. **Let a real user drive.**
7. **Finish with the measures,** what is still unknown, and what you recommend.

Keep a recording as a fallback in case the connection fails.

The decision paper (one to two pages)

- The question and the answer: yes, no, or not yet.
- Evidence against each success measure, and what users said in their own words.
- What it would take to run for real: people, cost range, risks, owner.
- Options: stop, extend the POC, run a pilot, or build for production.
- The recommendation and the decision needed, with a date.

TECHNICAL DETAIL

Monitoring and teardown

```
requests   | where success == false | summarize count() by name, resultCode,
bin(timestamp, 1h)
exceptions | order by timestamp desc | take 50
```

- Alerts: health check failing, HTTP 5xx, database CPU/DTU above 80 percent.
- Budget alert in Cost Management; confirm the tags are on every resource.
- Teardown: `az group delete -n $RG` removes everything inside the group. Check the name twice. Key Vault soft delete keeps the vault name reserved for its retention period.

PART 10

When things go wrong

Most problems are common and have known causes.

In plain words

What you see	Usual reason	What to do
"I can't create this"	A permission or policy limit	Ask IT which role or region is allowed (Part 5)
It worked this morning, now "access denied"	Time-limited access expired	Activate the role again
The site shows an error right after upload	The server is still starting or set up incorrectly	Wait a few minutes, then ask the builder to check the health page
Screens load but nothing saves	The screens and the back office are not connected	Builder checks the address and allowed origins
The bill is higher than planned	Something left running	List resources by cost; scale down or delete
Everyone wants one more feature	Scope creep	Return to the "won't this time" list and decision log
Testers don't use it	It solves a problem they don't have	Go back to Part 2: re-interview, adjust the problem

TECHNICAL DETAIL

Symptom	Cause and fix
503 after a zip deploy	Packages not installed. Set `ENABLE_ORYX_BUILD=true`
Startup script fails on Linux	CRLF line endings or backslash paths in the zip. Build the zip with a script
Deploy command hangs	Use `--async true`, poll `/health`
Browser "cannot reach server", API works in curl	CORS origin mismatch, or a SQLite connection shared across threads (`check_same_thread=False`)
Front end calls the wrong API	The address is baked in at build time and an old `.env.production` won. Set the variable explicitly; search the bundle for the host
`AuthorizationFailed` creating role assignments	Needs User Access Administrator or RBAC Administrator
Resource provider not registered	Subscription-level registration needed
Region or size not allowed	Policy; pick an allowed one
Key Vault reference not `Resolved`	Missing or stale role; redeploy to re-save settings
No database token on your laptop	Conditional access; use the in-app setup (Option B)
`az ad ...` fails	Graph blocked; use the token's `oid` claim, or read the id from inside the app
`FROM EXTERNAL PROVIDER` fails	Create the user from the identity's client id
PowerShell breaks `--query` with `[?...]`	Filter the JSON in PowerShell instead
Rules only in the UI	Anyone can call the API directly. Enforce on the server
Overwritten history	Keys that ignore the period (per unit instead of per unit per period). Fix before real data

PART 11

Checklist and timeline

Definition of done 0 of 16

- [ ] The question the POC answers is written down and agreed by the sponsor
- [ ] Interviews done with at least one person from each group; findings played back
- [ ] A one-page brief with a "won't this time" list exists
- [ ] Options compared (decision matrix) and the choice is explained
- [ ] Cost range estimated and verified in the Azure pricing calculator; budget alert set
- [ ] IT questions sent and answers recorded
- [ ] The main journey works end to end for every role
- [ ] The system refuses forbidden actions, and tests prove it
- [ ] Every change appears in the history
- [ ] Demo data covers every state
- [ ] Deployment is scripted; someone else can repeat it from the notes
- [ ] Health checks, logs and a budget alert exist
- [ ] Only fake data; no secrets in the code or repository
- [ ] Real users tried it and their words are recorded
- [ ] Success measures collected and the decision paper written
- [ ] The path to production is costed and has an owner

A four-week plan

Week	Focus	You should have
0	Frame and listen	The brief, interviews done, roles and workflow drawn, IT questions sent
1	Foundations	Records, rules, history, tests; an empty skeleton already on Azure
2	Main journey	The thin slice for every role, demo data, a first feedback session
3	Proper stage	Managed database, secrets in a vault, monitoring, insights views
4	Test and decide	Real users try it; measures collected; decision paper written

REFERENCE

Glossary

API

The back office of an application: the part that applies the rules and talks to the data.

Audit log

A record of every important change: who, what, before, after, when.

Azure

Microsoft's cloud: rentable computing, storage and services.

Bicep

A text file that describes the cloud resources you want, so they can be created repeatably.

CORS

A browser safety rule about which web addresses may talk to the back office.

Entra ID

Microsoft's company sign-in service.

Key Vault

A safe for passwords and keys.

Ledger

A database feature that makes after-the-fact changes to history detectable.

Managed identity

A badge the application itself holds, so it needs no stored password.

MoSCoW

Must, Should, Could, Won't: a way to rank features.

POC

Proof of concept: a small, time-boxed test of whether an idea is worth pursuing.

Resource group

A folder that holds everything for one project in Azure.

Role (RBAC)

A named set of permissions that can be given to a person or application.

SKU

The size or tier of a service, which sets its capability and price.

Tag

A label on a resource, such as owner or cost centre.

Thin slice

The smallest complete path through the system, built first.

WAF

Web application firewall: filters harmful traffic before it reaches the application.

Cost figures in this guide are rounded planning figures, not quotes. Check Azure services, limits and prices in official Microsoft documentation and the Azure pricing calculator before committing. LeaveLoop is a fictional example.